

Code Documentation

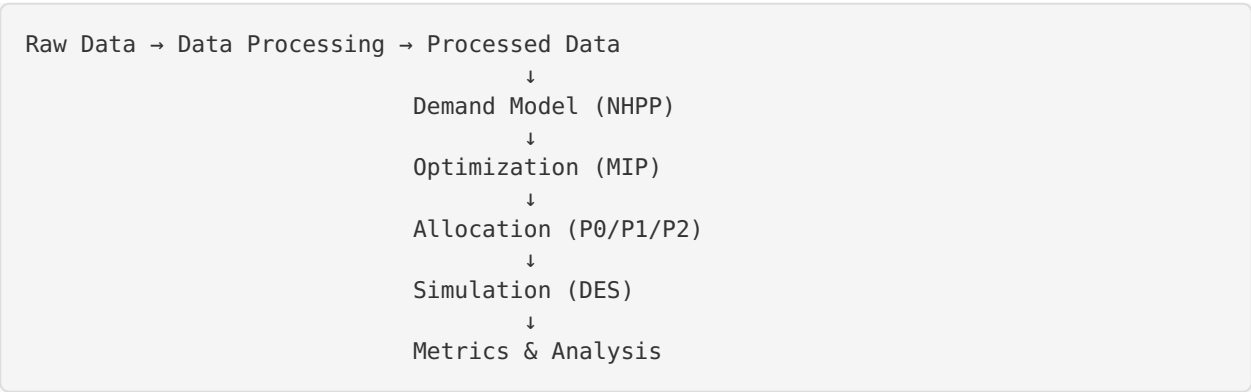
EMS Readiness Optimization — Architecture & Developer Guide

Version: 1.2.0 | Python: 3.11+ | Lines of Code: 7,800+

Architecture Overview

ems_readiness/	# Core package
├── __init__.py	# Package metadata (v0.5.0)
├── demand/	# Demand modeling module
│ ├── __init__.py	
│ └── arrival_generator.py	# NHPP arrival generation
├── service/	# Service modeling module
│ ├── __init__.py	
│ ├── travel_time.py	# Haversine-based travel proxy
│ └── service_time.py	# LogNormal service time
├── optimization/	# Optimization module
│ ├── __init__.py	
│ ├── models.py	# MIP formulations (PuLP)
│ ├── policies.py	# Baseline allocation policies
│ └── allocator.py	# High-level solver interface
├── simulation/	# DES module
│ ├── __init__.py	
│ ├── engine.py	# Main SimPy simulation
│ ├── entities.py	# Incident dataclass
│ ├── resources.py	# EMSUnit and UnitPool
│ ├── dispatcher.py	# Nearest-available dispatch
│ ├── metrics.py	# KPI collection
│ └── runner.py	# Batch experiment runner
└── utils/	# Utilities
├── __init__.py	
└── distance.py	# Haversine & Manhattan distance

Data Flow



Module Descriptions

demand.arrival_generator

Purpose: Generate non-homogeneous Poisson process arrivals using the Lewis-Shedler thinning algorithm.

Key Classes:

NHPPArrivalGenerator

```
class NHPPArrivalGenerator:
    def __init__(self, base_rate, hourly_factors, dow_factors, precinct_rates=None)

    @classmethod
    def from_tables(cls, data_dir, base_rate=3.48) -> NHPPArrivalGenerator

    def generate_arrivals(self, n_hours, start_hour=0, dow=0, rng=None) -> pd.DataFrame
```

Parameters:

- `base_rate` : Base arrival rate (calls/hour). Default: 3.48
- `hourly_factors` : `pd.Series` mapping hour → multiplier
- `dow_factors` : `pd.Series` mapping day-of-week → multiplier
- `precinct_rates` : Optional `pd.Series` for spatial allocation

Returns: `DataFrame` with columns `[time_hours, hour, precinct]`

Helper Functions:

- `load_lambda_tables(data_dir)` → tuple of `DataFrames`
 - `effective_rate(base_rate, hour, dow, hourly_factors, dow_factors)` → float
-

service.travel_time

Purpose: Convert Haversine distances to travel times with time-of-day speed adjustments.

Key Functions:

```
def travel_time_minutes(distance_miles, speed_mph=20.0, hour_of_day=None) -> float

def travel_time_from_coords(lat1, lon1, lat2, lon2, speed_mph=20.0, hour_of_day=None)
-> float

def build_travel_time_matrix(distance_matrix, speed_mph=20.0, hour_of_day=None) ->
pd.DataFrame
```

Constants:

- `DEFAULT_SPEED_MPH = 20.0`
 - `TOD_SPEED_FACTORS` : dict mapping hour → speed multiplier
-

service.service_time

Purpose: Sample on-scene service times from LogNormal distribution.

Key Class:

```
class ServiceTimeModel:
    def __init__(self, mean_minutes=25.0, std_minutes=10.0, distribution='lognormal')
    def sample(self, size=1, rng=None) -> np.ndarray
```

Supported distributions: 'lognormal', 'exponential'

optimization.models

Purpose: Define and solve MIP formulations using PuLP.

Key Functions:

```
def build_demand_weighted(travel_time_matrix, demand, K, capacity=5, ...) -> pulp.Lp-
Problem

def build_p_median(travel_time_matrix, demand, K, ...) -> pulp.LpProblem

def build_maximal_coverage(travel_time_matrix, demand, K, threshold_minutes=8.0, ...)
-> pulp.LpProblem

# --- Added in v1.2.0 (CBD-focused models) ---
def build_cbd_focused_demand_weighted(
    travel_time_matrix, demand, K, cbd_precincts, cbd_weight=3.0, capacity=5, ...
) -> pulp.LpProblem

def build_cbd_focused_coverage(
    travel_time_matrix, demand, K, cbd_precincts, threshold_minutes=8.0,
    cbd_coverage_weight=3.0, capacity=5, ...
) -> pulp.LpProblem

def extract_allocation(model) -> pd.Series
def extract_assignments(model) -> pd.DataFrame
def extract_coverage(model) -> pd.Series
```

Added in v1.2.0: `build_cbd_focused_demand_weighted()` applies a multiplier to CBD precinct demand weights, concentrating allocation toward the CBD. `build_cbd_focused_coverage()` applies analogous weighting to the coverage objective.

optimization.policies

Purpose: Non-optimized baseline allocation strategies.

```
def uniform_allocation(firehouses, K, capacity=5) -> pd.Series

def demand_proportional_allocation(travel_time_matrix, demand, K, capacity=5) -> pd.Se
ries
```

optimization allocator

Purpose: High-level interface orchestrating data loading, optimization, and evaluation.

```
class EMSAllocator:
    @classmethod
    def from_project(cls, project_root) -> EMSAllocator

    def solve(self, model='demand_weighted', K=40, ...) -> AllocationResult
    def baseline_uniform(self, K=40) -> AllocationResult
    def baseline_demand_proportional(self, K=40) -> AllocationResult
    def compare_models(self, K_values=[20,30,40,48]) -> pd.DataFrame
```

simulation engine

Purpose: Main DES event loop using SimPy.

```
class EMSSimulation:
    def __init__(self, allocation, distance_matrix, config, ...)

    @classmethod
    def from_config(cls, config_path, allocation) -> EMSSimulation

    def run(self, duration_hours=168, warm_up_hours=24, seed=None) -> dict
```

Simulation Process (per call):

1. Arrival generated by NHPP
 2. Dispatch delay (1.5 min fixed)
 3. Nearest available unit assigned
 4. Travel time (Haversine/speed × TOD factor)
 5. On-scene service (LogNormal sample)
 6. Return to home firehouse
-

simulation.entities

```
@dataclass
class Incident:
    id: int
    arrival_time: float
    precinct: int
    assigned_unit: str = None
    assigned_firehouse: str = None
    dispatch_time: float = None
    service_start_time: float = None
    completion_time: float = None
    travel_time_minutes: float = None
    service_time_minutes: float = None
    dispatch_delay_minutes: float = 1.5
    queued: bool = False

    @property
    def response_time_minutes(self) -> float

    @property
    def total_time_minutes(self) -> float
```

simulation.resources

```
class UnitStatus(Enum):
    AVAILABLE, DISPATCHED, ON_SCENE

@dataclass
class EMSUnit:
    id: str
    home_firehouse: str
    status: UnitStatus
    def dispatch(self) / arrive_on_scene(self) / return_available(self)

class UnitPool:
    def __init__(self, allocation: pd.Series)
    def get_available_units(self) -> list[EMSUnit]
    def count_available(self) -> int
    def get_utilizations(self) -> dict
```

simulation.dispatcher

```
class NearestAvailableDispatcher:
    def __init__(self, distance_matrix, speed_mph=20.0, use_time_of_day=True)

    def find_nearest_unit(self, precinct, unit_pool, hour_of_day=None) -> tuple[EMSUnit, float]
```

simulation.metrics

```
class MetricsCollector:
    def __init__(self, response_threshold_minutes=8.0)
    def record_incident(self, incident: Incident)
    def record_queue_length(self, time, length)
    def get_summary_statistics(self) -> dict
    def get_incident_log(self) -> pd.DataFrame
    def reset(self)
```

Summary statistics include: mean/median/P90/max response time, 8-min coverage fraction, mean utilization, queue metrics.

simulation.runner

```
class BatchRunner:
    def __init__(self, base_config, ...)

    def run_experiment(self, allocation, n_reps=30, ...) -> pd.DataFrame
    def run_factorial(self, factors, ...) -> pd.DataFrame
```

Features: CRN support, parallel execution, progress tracking, result aggregation.

utils.distance

Purpose: Geographic distance calculations with support for Haversine and Manhattan metrics.

```
EARTH_RADIUS_MILES = 3958.8
MILES_PER_DEGREE_LAT = 69.0
MILES_PER_DEGREE_LON_NYC = 52.3 # At 40.75°N latitude

def haversine(lat1, lon1, lat2, lon2) -> float # great-circle distance (miles)

def manhattan_distance(lat1, lon1, lat2, lon2) -> float # taxicab distance (miles)

def build_distance_matrix(origins, destinations, ..., metric="haversine") -> pd.DataFrame
    # metric: "haversine" or "manhattan"
```

Added in v1.2.0: `manhattan_distance()` function and `metric` parameter for `build_distance_matrix()`. Manhattan distance computes $|\Delta\text{lat}| \times 69.0 + |\Delta\text{lon}| \times 52.3$ miles, approximating grid-based travel in NYC.

Configuration Guide

configs/demand.yaml

```
base_rate_per_hour: 3.48
lambda_tables:
  hourly: data/processed/demand_lambda_hourly.csv
  dow: data/processed/demand_lambda_dow.csv
  precinct: data/processed/demand_lambda_precinct.csv
simulation:
  default_duration_hours: 168
  default_replications: 30
  seed: 42
```

configs/service.yaml

```
travel_time:
  average_speed_mph: 20.0
  use_time_of_day: true
service_time:
  distribution: lognormal
  mean_minutes: 25.0
  std_minutes: 10.0
dispatch_delay_minutes: 1.5
```

configs/optimization.yaml

```
unit_counts: [20, 30, 40, 48]
firehouse_capacity: 5
coverage_threshold_minutes: 8.0
solver:
  name: CBC
  time_limit_sec: 300
```

Extension Guide for Future Developers

Adding a New Allocation Policy

1. Add policy function in `optimization/policies.py` :

```
def my_new_policy(travel_time_matrix, demand, K, **kwargs) -> pd.Series:
    # Return pd.Series: firehouse_name -> unit_count
    pass
```

1. Register in `optimization/allocator.py` `solve()` method
2. Add to `configs/optimization.yaml` `compare_models` list

Adding a New Dispatch Strategy

1. Create new dispatcher class in `simulation/dispatcher.py` :

```
class MyDispatcher:
    def find_nearest_unit(self, precinct, unit_pool, hour_of_day=None):
        # Return (EMSUnit, travel_time_minutes)
        pass
```

1. Pass to `EMSSimulation.__init__(dispatcher=MyDispatcher(...))`

Adding a New Experiment

1. Define factor levels in a new script:

```
factors = {
    'policy': ['P0', 'P1', 'P2'],
    'my_factor': [val1, val2, val3]
}
runner = BatchRunner(base_config)
results = runner.run_factorial(factors, n_reps=30)
```

1. Save results to `results/simulation/production/`
2. Add analysis to `scripts/analyze_production_results.py`

Integrating Road Network Routing

1. Replace `haversine()` with OSRM API calls in `utils/distance.py`
2. Update `build_distance_matrix()` to use routing distances
3. Rebuild `distance_matrix_firehouse_precinct.csv`
4. No changes needed in optimization or simulation code

Dependencies

Package	Version	Purpose
pandas	≥2.0	Data manipulation
numpy	≥1.24	Numerical computing
simpy	≥4.0	Discrete-event simulation
pulp	≥2.7	Linear/integer programming
scipy	≥1.11	Statistical functions
matplotlib	≥3.7	Visualization
seaborn	≥0.12	Statistical visualization
geopandas	≥0.13	Geospatial data
pytest	≥7.0	Testing

See `requirements.txt` for complete dependency list.

Last updated: March 12, 2026